What if some of the salary values were identical?

= What if an employee had signed two or more contracts of the same value, albeit related to different periods?

In this case, it would be worth considering the use of the rank and dense rank window functions.

RANK() and DENSE_RANK()

- Window Functions in MySQL:
  - They all require the use of the OVER clause
  - The rank values they provide are always assigned sequentially
  - The first rank is always equal to the integer *1*, and the subsequent rank values grow *incrementally by 1*, except for the duplicate records potentially

RANK() and DENSE_RANK()

- Window Functions in MySQL:
  - RANK() and DENSE_RANK() are only useful when applied on *ordered partitions* (=partitions defined by the use of the ORDER BY clause)

	type:	ORDER BY
ROW_NUMBER()	non-order-sensitive	not necessarily
RANK() DENSE_RANK()	order-sensitive	more meaningful

Don't Limit

```
USE employees;

SELECT
    emp_no, salary, RANK() OVER w AS rank_num
FROM
    salaries
WHERE emp_no = 11839
WINDOW w AS (PARTITION BY emp_no ORDER BY salary DESC);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_no	salary	rank_num
11839	95810	1
11839	93590	2
11839	89814	3
11839	89814	3
11839	89755	5
11839	85488	6

	the focus is on:
RANK()	the *number* of values we have in our output
DENSE_RANK()	the *ranking* of the values *itself*

```sql
SELECT
    emp_no, salary, DENSE_RANK() OVER w AS rank_num
FROM
    salaries
WHERE emp_no = 11839
WINDOW w AS (PARTITION BY emp_no ORDER BY salary DESC);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_no	salary	rank_num
11839	95810	1
11839	93590	2
11839	89814	3
11839	89814	3
11839	89755	4
11839	85488	5

Notice how the DENSE RANK function acknowledges the value itself so increments the rank precisely being so its 3,3, then 4. Whereas Rank only acknowledges the position of each value rather than the substance of each value, it ended up being 3,3 then 5.

The MySQL RANK() and DENSE_RANK() Window Functions - Exercise #2

Order and rank all contract salary values of employee 10002 from highest to lowest. Store the row numbers in a third column named `order_num`. Assign the same rank to identical salary values allowing gaps in the obtained ranks for subsequent rows.

To obtain the desired values, refer to the employee number (`emp_no`) and salary (`salary`) columns from the `salaries` table.

```sql
SELECT
emp_no,
salary,
RANK() OVER w AS order_num
FROM
salaries
WHERE emp_no = 10002
WINDOW w AS (PARTITION BY emp_no ORDER BY SALARY DESC);
```

Run query | Reset | All changes saved | Line 1, Column 1

Result | Success

The MySQL RANK() and DENSE_RANK() Window Functions - Exercise #3

Order and rank all contract salary values of employee 10002 from highest to lowest. Store the row numbers in a third column named `order_num`. Assign the same rank to identical salary values *without* allowing gaps in the obtained ranks for subsequent rows.

To obtain the desired values, refer to the employee number (`emp_no`) and salary (`salary`) columns from the `salaries` table.

```sql
SELECT
emp_no,
salary,
DENSE_RANK() OVER w AS order_num
FROM
salaries
WHERE emp_no = 10002
WINDOW w AS (PARTITION BY emp_no ORDER BY SALARY DESC);
```

Run query | Reset | All changes saved | Line 8, Column

Result | Success